

CS214-Data Structure

Lecturer: Dr. Salwa Osama

Linked Lists Use Cases

Types of Linked List

- There are three common types of Linked List.

1. Singly Linked List

2. Doubly Linked List

3. Circular Linked List

Singly Linked List

- It is the most common. Each node has data and a pointer to the next node.



```
struct node {  
    int data;  
    struct node *next;  
}
```

Doubly Linked List

- We add a pointer to the previous node in a doubly-linked list. Thus, we can go in either direction: forward or backward.



```
struct node {  
    int data;  
    struct node *next;  
    struct node *prev;  
}
```

Circular Linked List

- A circular linked list is a variation of a linked list in which the last element is linked to the first element. This forms a circular loop.



- A circular linked list can be either singly linked or doubly linked.
 - for singly linked list, next pointer of last item points to the first item
 - In the doubly linked list, prev pointer of the first item points to the last item as well.

Your Mission

- Implement six operations of the circular single linked list
- Implement six operations of the double linked list.

```
typedef char    Entrytype;  
typedef struct  node {  
    Entrytype    info;  
    struct node  *next;  
}Node;  
typedef Node *  ListType;
```

Having the above declaration of linked list:

- 1) Write a C function to print all list elements.
- 2) Write a C function to print all list elements in reverse order.

```
#include "stack.h"
```

```
void printRev ListType L){
```

```
    Node *p = L
```

```
    stackType s;        createStack(&s);
```

```
    while (p && !fullStack(s)){
```

```
        push(&s, p->info)           p=p->next;
```

```
    }
```

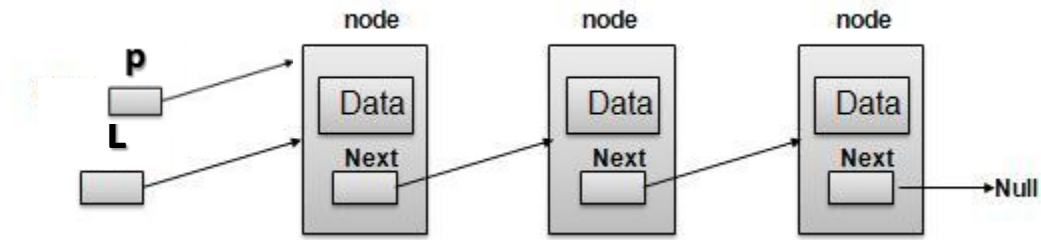
```
    Entrytype c;
```

```
    while (!emptyStack(s)){
```

```
        pop(&s,&c);    printf("%c", c);
```

```
    }
```

```
}
```



```
typedef char Entrytype;
```

```
typedef struct node {
```

```
    Entrytype    info;
```

```
    struct node  *next;
```

```
}Node;
```

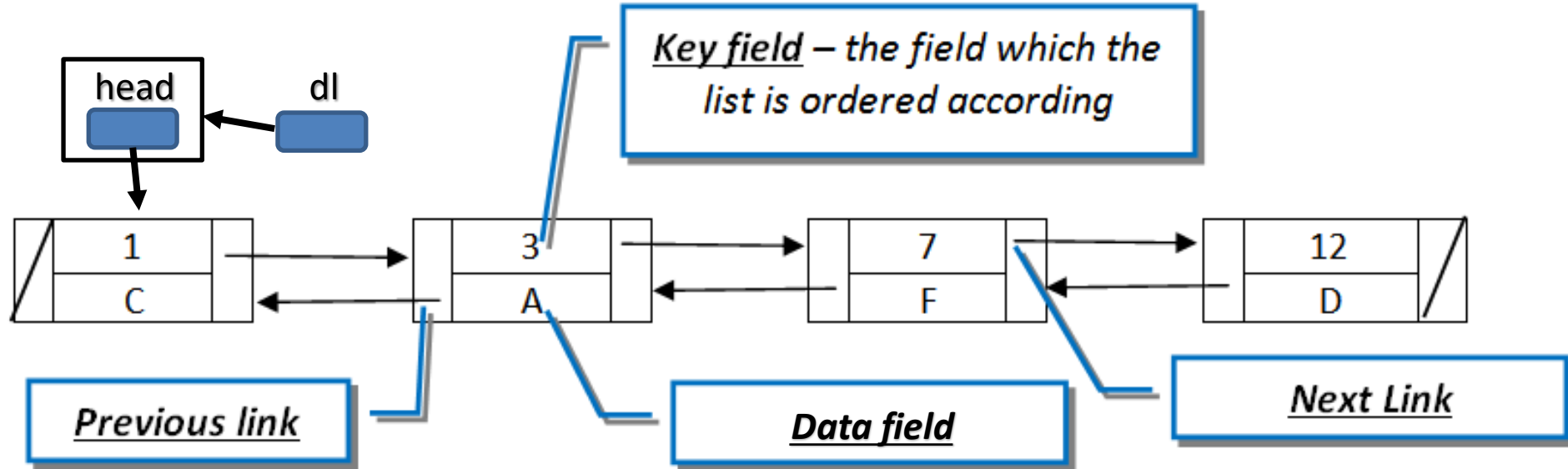
```
typedef Node * ListType;
```

A **doubly sorted linked list** is a linked data structure that consists of a set of sequentially linked nodes. Data in the list should be **sorted according specific key**. Each node contains:

- a data entry field that contain a **key** and **information**,
- and **two pointers** that are references to the previous and to the next node in the

You are required to:

1. Write the type definition of a doubly sorted linked list node in C.
2. Write a C function to initialize the list.
3. Write a C function to insert an element.
4. Write a C function to delete an element of specific key after retrieving its information.



```
typedef struct{
    int    Key;
    char   Data;
}eType;
```

```
typedef struct{
    node * head;
}dLType;
```

```
typedef struct nodes {
    eType      info;
    struct nodes *next;
    struct nodes *prev;
}node;
```

```
void createDL(dLType* dl) {
    dl->head = null;
};
```

```
void insert(dLType *dl, eType e){
```

```
    node * p= (node*)malloc(sizeof(node));
```

```
    p->info = e;
```

```
    node * q=dl->head;
```

```
    if (dl->head ==null){
```

```
        p->prev = null;
```

```
        p->next = null ;
```

```
        dl->head = p;
```

```
    }else{
```

```
        q=q->next;
```

```
        p->next = q;
```

```
        p->prev=q->prev;
```

```
        q->prev->next =p;
```

```
        q->prev = p;
```

```
    }}
```

